Массивы. Объекты

Занятие 6

Ход занятия

1. Массивы.
2. Объекты.
3. Практика.

Объекты

Объекты позволяют хранить данные со строковыми ключами в неупорядоченном виде.

```
let user = {
    name: "Ivan",
    age: 18
};
```

Но довольно часто мы понимаем, что нам необходима упорядоченная коллекция данных, в которой присутствуют 1-й, 2-й, 3-й элементы - список.

Массив

Массив – это особый тип объекта, предназначенный для работы с упорядоченным набором элементов.

В JavaScript индексы начинаются с 0. [‘apple’, 1, true, null]

Устройство массива

Массивы расширяют объекты, так как предусматривают специальные методы для работы с упорядоченными коллекциями данных.

Но массив нежелательно использовать как обычный объект.

Массивы тщательно настроены в движках JavaScript для работы с однотипными упорядоченными данными.

Если нужны произвольные ключи, лучше подойдёт обычный объект.

Определить тип

Метод Array.isArray(value):

```
alert(typeof {}); // object
alert(typeof []); // тоже object
```

Проверка на массив:

```
alert(Array.isArray({})); // false
alert(Array.isArray([])); // true
```

Объявление

Существует два варианта синтаксиса для создания пустого массива:

```
let arr = new Array();

let arr = [];
```

* Рекомендуется использовать второй способ [] самый короткий.

Особенность! Если new Array вызывается с одним аргументом, который представляет собой число, он создает массив без элементов, но с заданной длиной.

Работа с массивом

Чтобы получить доступ к свойству используется синтаксис []

```
obj[key] //в качестве ключа key – числовой индекс

let fruits = ["Яблоко", "Апельсин"];

alert(fruits[0]); //Яблоко
```

1. Можно заменить любой элемент
2. Элементом массива может быть что угодно

Примеры

```
let numbers = [1, 2, 3, 4];

let fruits = ["Яблоко", "Апельсин", "Слива"];

let arr = [ 'Яблоко', { name: 'Джон' }, true, function() { alert('привет'); }
];


alert( arr[1].name ); // Джон

arr[3]() ; // привет
```

Многомерные массивы

```
let matrix = [
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9]
];
alert( matrix[0][1] ); // 5, центральный элемент
```

Неправильное использование массива

- Добавление нечислового свойства, например: arr.test = 5. Теряем всю оптимизацию, JavaScript перестает работать как с массивом.
- Создание «дыр», например: добавление arr[0], затем arr[1000] (между ними ничего нет).
- Заполнение массива в обратном порядке, например: arr[1000], arr[999] и т.д.

Свойства массива

Свойство length - это не количество элементов массива, а наибольший цифровой индекс плюс один.

```
let fruits = [];

fruits[123] = "Яблоко";

alert( fruits.length ); // 124
```

length – можно перезаписать, но не стоит этого делать, массив в таком случае безвозвратно станет короче по заданному length, arr.length = 0 очистит массив.

Приведение массива к строке

Массивы по своему реализуют метод toString(), который возвращает список элементов разделенных запятой.

```
let arr = [1, 2, 3];

alert( String(arr) === '1,2,3' ); // true

arr.toString(); //"1,2,3"
```

Методы добавления \ удаления элементов

push - Добавляет элемент в конец, и возвращает новую длину массива.

unshift - Добавляет элемент в начало массива, и возвращает новую длину массива.

pop - Удаляет последний элемент из массива и возвращает его.

shift - Удаляет из массива первый элемент и возвращает его.

Методы push и unshift могут добавлять сразу несколько элементов.

Как это работает

- Методы push/pop выполняются быстро, а методы shift/unshift – медленно.
- Операции с началом массива вызывают перенумерацию.
- Например просто взять и удалить элемент с номером 0 недостаточно. Нужно также заново пронумеровать остальные элементы.
- Чем больше элементов содержит массив, тем больше времени потребуется.

Методы поиска в массиве

Методы arr.indexOf(), arr.includes() имеют одинаковый синтаксис и делают по сути то же самое, что и их строковые аналоги, но работают с элементами вместо символов:

- arr.indexOf(find, start) ищет find, начиная с индекса start, и возвращает индекс, на котором был найден искомый элемент, в противном случае -1.
- arr.includes(find, start) – ищет find, начиная с индекса start, и возвращает true, если поиск успешен.

Полезные методы - find()

Метод find(), возвращает значение первого найденного в массиве элемента, удовлетворяющее условию, заданному в функции. В противном случае вернет undefined.

```
let arr = [1, 2, 3, 4];

let findElem = arr.find((e) => e  === 3);

console.log(findElem); // 3
```

Полезные методы - filter()

Метод filter(), создает новый массив из значений удовлетворяющих условию, заданному в функции.

```
let arr = [1, 2, 3, 4];

let filterArr = arr.filter((e) => e % 2 === 0);

console.log(filterArr); // [2, 4]
```

Полезные методы - map()

Метод map(), создает новый массив с результатом вызова переданной функции к каждому элементу

```
let arr = [1, 2, 3, 4];

let mapArr = arr.map((e) => e * 10);

console.log(mapArr); // [10, 20, 30,  40]
```

Полезные методы - forEach()

Метод forEach(), применяет переданную функцию к каждому элементу один раз.

```
let arr = [1, 2, 3, 4];

arr.forEach((e) => console.log(e));

// 1;

// 2;

// 3;

// 4;
```

Полезные методы - reduce()

Метод reduce(), применяет переданную функцию к каждому элементу массива и возвращает результирующее значение.

```
let arr = [1, 2, 3, 4];

let result = arr.reduce((accum, current) => accum + current);

console.log(result); // 10
```

Практика

- Написать функцию, которая будет принимать массив чисел, содержащий целые положительные и целые отрицательные числа, в качестве результата возвращать сумму четных положительных элементов переданного массива.
- Написать функцию, которая будет принимать массив чисел, и будет убирать повторяющиеся значения из переданного массива, в качестве результата возвращать новый массив с уникальными значениями из исходного.
- Написать функцию которая будет принимать два массива, и будет сравнивать их, если они идентичны (элементы совпадают по значению и по индексу) то функция возвращает true, в противном случае false.

Копирование объектов

```
let firstObj = { a : 'Hello' };
let secondObj = firstObj;
console.log(firstObj , secondObj ); // { a : 'Hello' }, { a : 'Hello' };

firstObj.a = 'Goodbye';
console.log(firstObj , secondObj ); // { a : 'Goodbye' }, { a : 'Goodbye' };
```

Важно запомнить, что объекты копируются по ссылке. Изменяя один объект который ссылается на выделенную область памяти, будет изменяться и другой который ссылается на эту же область в памяти.

Решение

Создать новый объект который будет ссылаться на новую область в памяти.

```
Object.assign( target, ... sources);
```

target - целевой объект в который произойдет копирование.

sources - объекты которые будут скопированы.

```
let firstObj = { a : 'Hello' };

let secondObj = Object.assign( { b: 'Hi!' }, firstObj);
console.log(firstObj , secondObj ); // { a : 'Hello' }, { b: 'Hi!', a : 'Hello' };
```

Как это работает

- Метод Object.assign(target, ... sources) создает новый объект в области памяти.
- Количество копируемых исходных объектов неограниченно.
- Копирует свойства всех исходных объектов sources, в объект target. То есть свойства всех объектов начиная со второго копируются в первый.
- Возвращается объект target.

Оператор расширения (spread)

Оператор spread позволяет расширить доступные для итерации элементы (например, массивы, строки) в местах:

- для функций: где ожидаемое количество аргументов для вызовов функций равно нулю или больше нуля
- для элементов (литералов массива)
- для выражений объектов: в местах, где количество пар "ключ-значение" должно быть равно нулю или больше (для объектных литералов)

Оператор расширения (spread)

```
let arr = [1, 2, 3];

let newArr = [... arr, 4, 5, 6];

console.log(newArr); // [1, 2, 3, 4, 5, 6]

let obj = { a : 'Hello'};

let newObj = { ... obj }; // новый объект, который будет ссылаться на
новую область памяти

newObj.b = 'Hi!';

console.log( obj, newObj); // { a : 'Hello'}, { a : 'Hello', b: 'Hi!'}
```

Остаточные параметры (rest оператор)

Оператор rest позволяет группировать неограниченное количество остаточных аргументов функции в массив.

- Используется для создания функций с неопределенным количеством аргументов.

```
let example = ( a, b, ... args) => {
    console.log(args); // массив остаточных аргументов
}
```

Перебор значений массива

Для перебора значений мы можем использовать обычный цикл for(...), но в JavaScript есть специальный более короткий синтаксис for (... of ...):

```
let arr = [1, 2];

for (let variable of arr) {
    console.log(variable * 10 );

}

// 10;

// 20;
```

Перебор значений объекта

Для перебора значений объекта мы можем использовать цикл for (... in ...):

```
let obj = { a : 1, b : 2};

for (let variable in obj) {
        console.log(obj[variable] * 10 );

}

// 10;

// 20;
```

Практика

- Создайте объект и скопируйте данный объект с помощью: Object.assign() и spread оператора. Изменить любое свойство в копии объекта, и проверить не изменился ли исходный.
- Написать функцию которая будет принимать n-ое количество аргументов, в качестве результата функция будет возвращать сумму всех четных элементов. Для решения использовать цикл for (... of ...).
- Написать функцию которая будет принимать два массива, и в качестве результата будет возвращать только те значения которые есть и в первом и во втором массиве.